

PRÁCTICAS PROFESIONALIZANTES 2026

Escuela de Artes Visuales Martín A. Malharro

INFORME TÉCNICO — EQUIPO B

Deployment, Infraestructura y Mantenimiento

Integrantes:

Salvia Malena

Soto Valentina

Ledesma Antonella

Foresi Gaia

Docente a cargo: Tresser Lucas

Mar del Plata, Buenos Aires, Argentina — 2026

Introducción

Descripción del proyecto

El sistema web institucional de la Escuela de Artes Visuales Martín A. Malharro se encuentra desarrollado con las siguientes tecnologías: Next.js, React, Node.js, PostgreSQL, GNU/Linux y PM2.

Rol del Equipo B

El Equipo B tiene a su cargo las tareas vinculadas al deployment, la infraestructura y el mantenimiento del servidor. Sus responsabilidades incluyen el acceso y la administración del servidor GNU/Linux, la documentación del deployment cloud, la configuración y gestión de PM2, la elaboración de manuales de mantenimiento y recuperación, el registro de variables de entorno y la gestión de backups.

Objetivos

- Documentar el acceso al servidor mediante SSH.
- Registrar la configuración del servidor y el entorno de producción.
- Documentar las variables de entorno y procedimientos de backup.
- Elaborar el tutorial completo de deployment en servidor cloud.
- Configurar y documentar PM2 para la gestión del proceso.
- Elaborar el manual de mantenimiento y recuperación del servidor.

Estructura del documento

El presente informe se organiza en cinco etapas que reflejan el desarrollo cronológico de la práctica. Cada etapa describe las actividades realizadas, los procedimientos aplicados y los entregables generados por el Equipo B.

Etapa 1 — Relevamiento y Análisis del Servidor

La Etapa 1 corresponde al relevamiento inicial del servidor GNU/Linux en el que se encuentra desplegado el sistema institucional. El objetivo de esta etapa es obtener un panorama completo del estado del servidor, los servicios activos y la configuración existente de PM2, a fin de contar con una base documentada para las etapas siguientes.

Análisis del Servidor GNU/Linux

Información general del servidor:

Campo	Valor
Sistema operativo	Ubuntu 24.04.4 LTS (Noble Numbat)
RAM total	7,8 GB totales — 1,2 GB en uso — 6,6 GB disponibles
Espacio en disco (/)	39 GB totales — 14 GB en uso — 26 GB disponibles (36% utilizado)
Versión de Node.js	v18.19.1
Versión de npm	9.2.0
Versión de PM2	6.0.13
Versión de PostgreSQL	PostgreSQL 16.13 (Ubuntu 16.13-0ubuntu0.24.04.1)
Hostname	vps-5403081-x
IP del servidor	168.181.184.177
Usuario SSH	tecnica5

Verificación de la Configuración de PM2

Estado de los procesos PM2

Al ejecutar el comando `pm2 list` se obtuvo una tabla vacía sin procesos listados. No se encontraron aplicaciones Node.js corriendo bajo PM2 al momento del relevamiento.

id	name	mode	↻	status	cpu	memory
(sin procesos)						

Configuración del ecosistema PM2

El archivo `ecosystem.config.js` no fue encontrado en la ruta del proyecto: `/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/`. Tampoco existía la variante `ecosystem.config.cjs`. La aplicación no estaba configurada para administración mediante PM2 a través de un archivo de ecosistema.

Relevamiento de la Estructura de Deployment

El proyecto se encuentra alojado en: `/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/`

Archivos y directorios identificados:

Archivo / Directorio	Tamaño	Observación
backend/	—	Directorio backend de la aplicación.
malharrooficial/	—	Sub-sitio o proyecto adicional.
package.json	—	Proyecto: my-app v0.1.0. Next.js 15.5.9, React 19, Node.js.
data.log	1,73 GB	Archivo de log de gran tamaño. Requiere rotación.
monitor.log	38 KB	Log de monitoreo activo. Última modificación: 13 de mayo.
failed.log	462 B	Registro de fallos del sistema.
xmrig-6.21.0 / xmrig.tar.gz	—	⚠ Software de minería de criptomonedas. Presencia sospechosa.
scanner_linux	0 B	⚠ Ejecutable de escáner. Origen y propósito a verificar.
exploited.log	813 B	⚠ Nombre altamente sospechoso. Posible indicador de intrusión.

Identificación de Servicios Activos

Servicio / Puerto	Descripción
Nginx (puerto 80)	Servidor web y proxy inverso.
MariaDB	Motor de base de datos relacional.
PostgreSQL (instalado)	Base de datos relacional v16.13. No se detectó puerto abierto explícito.
PHP-FPM (7.1–8.3)	Procesadores PHP en múltiples versiones para compatibilidad.
Redis	Base de datos en memoria / caché.
Fail2ban	Seguridad: bloqueo de IPs con intentos fallidos.
SSH (puertos 22 y 5174)	Acceso remoto seguro. Puerto alternativo 5174 configurado.
Postfix (puerto 25)	Servidor de correo SMTP.
ProFTPD (puerto 21)	Servidor FTP.
CloudPanel (puerto 8443)	Panel de administración web (HTTPS).
Memcached / Varnish (6081)	Caché de objetos en memoria.
Cron	Programación de tareas automáticas.
Node.js / Next.js (PM2)	No activo al momento del relevamiento.

Observaciones y Hallazgos del Relevamiento

1. La aplicación Next.js no está en ejecución. PM2 no registra ningún proceso activo. La aplicación deberá iniciarse con `pm2 start` y configurarse con un archivo `ecosystem.config.js`.
2. Presencia de xmrig. El software xmrig es un minero de criptomonedas de código abierto. Su presencia en un servidor web es una señal de posible compromiso de seguridad (cryptojacking). Se recomienda investigación inmediata.
3. Archivos `exploited.log` y `scanner_linux`. Estos archivos sugieren que el servidor pudo haber sido comprometido. Requieren revisión urgente.
4. `data.log` de 1,73 GB. Ocupa espacio significativo en disco. Se recomienda evaluar rotación o limpieza de logs.
5. No se contaba con documentación técnica previa del servidor. La configuración fue relevada manualmente mediante herramientas de consola.

Estado del sistema al inicio de la práctica

El sistema presentaba carga baja y funcionamiento estable al momento del relevamiento. No se detectaron problemas de rendimiento ni consumo excesivo de recursos, más allá de los hallazgos de seguridad descritos anteriormente.

Etapa 2 — Documentación Técnica

La Etapa 2 tiene como objetivo transformar los datos relevados en la etapa anterior en documentación técnica formal y estructurada. Se documenta el acceso SSH, la configuración del servidor, las variables de entorno, la configuración de PM2, los procesos de backup y el registro de logs.

Documentación de Acceso SSH

Parámetro	Valor / Detalle
Comando de conexión	ssh -p 5174 tecnica5@168.181.184.177
Usuario	tecnica5
Servidor	168.181.184.177
Puerto	5174
Tipo de autenticación	Contraseña (password)
Claves SSH en el servidor	Ninguna. El archivo authorized_keys está vacío.
Archivo ~/.ssh/config	Existe en el servidor pero no contiene entradas definidas.

Configuración del Servidor

Datos del entorno de producción:

Campo	Valor
hostname	vps-5403081-x
IP pública	168.181.184.177
Sistema operativo	Ubuntu 24.04.4 LTS (Noble Numbat)
Zona horaria	America/Argentina/Buenos_Aires (-03, -0300)
NTP / Sincronización	Activo — Reloj sincronizado
Usuario SSH activo	tecnica5

Versiones de software instalado:

Software	Versión instalada
Node.js	v18.19.1
npm	9.2.0
Git	git version 2.43.0
Nginx	nginx/1.30.0
PostgreSQL	16.13 (Ubuntu 16.13-0ubuntu0.24.04.1)
PM2	6.0.13

Configuración del Servidor Web (Nginx / Proxy Inverso)

El servidor utiliza Nginx 1.30.0 como proxy inverso, gestionado mediante CloudPanel v2. El archivo de configuración principal se encuentra en `/etc/nginx/nginx.conf`. Las configuraciones específicas de cada sitio (virtual hosts) se ubican en `/etc/nginx/sites-enabled/` e incluidas automáticamente mediante la directiva: `include /etc/nginx/sites-enabled/*.conf;`

Parámetros generales relevados:

Parámetro	Valor	Descripción
<code>worker_connections</code>	2000	Conexiones simultáneas máximas
<code>client_body_size</code>	10m	Tamaño máximo de cuerpo en requests
<code>client_header_buffer_size</code>	1k	Buffer de cabeceras HTTP
<code>keepalive_timeout</code>	60	Timeout de conexiones persistentes
<code>ssl_protocols</code>	TLSv1.2, TLSv1.3	Protocolos SSL/TLS habilitados
<code>ssl_session_timeout</code>	10m	Duración de sesión SSL

Logs de acceso y error:

Tipo	Ruta
Access log	<code>/var/log/nginx/access.log</code>
Error log	<code>/var/log/nginx/error.log</code>
Logs rotados	<code>error.log.1</code> a <code>error.log.14.gz</code>

Gestión de la configuración: La administración del proxy inverso, los virtual hosts y los certificados SSL se realiza a través del panel web CloudPanel, accesible en: <https://168.181.184.177:8443>

NOTA: El usuario `tecnica5` no tiene acceso directo de lectura a `/etc/nginx/sites-enabled/` por política de permisos del servidor. La configuración de cada sitio es gestionada exclusivamente por el administrador desde CloudPanel.

Documentación de Variables de Entorno

.env.local — Frontend (Next.js):

Variable	Propósito	Formato esperado
<code>PORT</code>	Puerto en que corre el frontend	Número entero (ej: 3000)
<code>NEXT_PUBLIC_API_URL</code>	URL base de la API del backend	<code>http://IP:puerto</code>

.env.production — Frontend producción:

Variable	Propósito	Formato esperado
<code>NEXT_PUBLIC_API_URL</code>	URL pública de la API en producción	<code>http://IP</code>

backend/.env — Backend (Node.js):

Variable	Propósito	Formato esperado
PGHOST	Host de PostgreSQL	localhost o IP
PGPORT	Puerto de PostgreSQL	Número entero (ej: 5432)
PGDATABASE	Nombre de la base de datos	String (ej: nombre_db)
PGUSER	Usuario de PostgreSQL	String
PGPASSWORD	Contraseña de PostgreSQL	String seguro
DATABASE_URL	Cadena de conexión completa a PostgreSQL	postgresql://user:pass@host:port/db
PORT	Puerto en que corre el backend	Número entero (ej: 4000)
NODE_ENV	Entorno de ejecución	production / development
CLIENT_URL	URL del frontend permitida (CORS)	http://IP o dominio
JWT_SECRET	Clave secreta para firmar tokens JWT	String aleatorio largo
JWT_EXPIRES_IN	Tiempo de expiración del token JWT	Duración (ej: 8h)
ADMIN_EMAIL	Email del administrador del sistema	Dirección de correo válida
ADMIN_PASSWORD	Contraseña del administrador	String seguro
SMTP_HOST	Servidor SMTP para envío de emails	Dominio (ej: smtp.gmail.com)
SMTP_PORT	Puerto SMTP	Número (ej: 465)
SMTP_SECURE	Usar conexión segura en SMTP	true / false
SMTP_USER	Usuario de la cuenta SMTP	Dirección de correo
SMTP_PASS	Contraseña o app password SMTP	String
EMAIL_FROM	Nombre y dirección del remitente	"Nombre" <correo@dominio>

Gestión del .env y permisos:

Archivo	Permisos	Propietario
backend/.env	-rw-rw---- (660)	u4395av_mm
.env.local	-rw-rw---- (660)	u4395av_mm
.env.production	-rw-rw---- (660)	u4395av_mm

Los archivos son legibles solo por el propietario y su grupo. No son accesibles públicamente. Las variables de entorno son cargadas automáticamente por Next.js (.env.local, .env.production) y por el proceso Node.js del backend al iniciarse.

Registro de la Configuración de PM2

Estado de procesos: Al momento del relevamiento, pm2 list no muestra procesos activos. El sistema no tiene servicios corriendo bajo PM2 en este momento.

Archivo de ecosistema: No existe archivo ecosystem.config.js en el proyecto. El inicio de procesos se realizaría mediante comandos pm2 start directos.

Startup automático: El sistema usa systemd como init system. El comando para configurar el inicio automático de PM2 es:

```
sudo env PATH=$PATH:/usr/bin /usr/local/lib/node_modules/pm2/bin/pm2 startup  
systemd -u tecnica5 --hp /home/tecnica5
```

Estado: El startup automático está configurado a nivel de sistema pero sin procesos guardados (pm2 save no tiene procesos para persistir).

Historial de comandos PM2 relevados (del .bash_history):

```
pm2 -v                # verificar versión instalada  
pm2 --version          # verificar versión instalada  
pm2 list               # listar procesos activos  
pm2 show [Nombre_o_ID] # ver detalle de un proceso  
pm2 logs --list        # listar archivos de log disponibles
```

Estructura interna de PM2:

```
~/ .pm2/  
├─ logs/                # archivos de log de procesos  
├─ module_conf.json  
├─ modules/  
├─ pids/  
├─ pm2.log              # log interno de PM2  
├─ pm2.pid  
├─ pub.sock  
├─ rpc.sock  
└─ touch
```

Documentación de Backups y Logs

Crontab: No hay tareas programadas configuradas para el usuario tecnica5. El backup no está automatizado.

Carpeta de backups: Existe el directorio ~/backups/ con las siguientes subcarpetas:

```
~/backups/  
└─ databases/          # backups de base de datos
```

Backup manual de PostgreSQL:

```
pg_dump -U malharro_user -h localhost -p 5432 malharro_db \  
> ~/backups/databases/backup_$(date +%Y%m%d_%H%M%S).sql
```

Restauración de la base de datos:

```
psql -U malharro_user -h localhost -p 5432 malharro_db \  
< ~/backups/databases/backup_FECHA.sql
```

Logs de PM2:

```
~/ .pm2/logs/          <- vacío al momento del relevamiento  
~/ .pm2/logs/nombre-proceso-out.log  # salida estándar  
~/ .pm2/logs/nombre-proceso-error.log # errores
```

Comandos de consulta:

```
pm2 logs              # tiempo real  
pm2 logs --list       # listar archivos disponibles
```

Logs de Nginx:

Tipo	Ruta
Access log	/var/log/nginx/access.log
Error log	/var/log/nginx/error.log
Logs rotados	error.log.1 a error.log.14.gz

Cómo consultarlos:

```
tail -f /var/log/nginx/access.log    # tiempo real
tail -f /var/log/nginx/error.log     # errores en tiempo real
cat /var/log/nginx/error.log         # log completo
```

Rotación de logs — Nginx (configurado en /etc/logrotate.d/nginx):

Parámetro	Valor	Significado
Frecuencia	daily	Rota una vez por día
rotate	14	Conserva 14 archivos históricos
compress	sí	Comprime logs rotados con gzip
delaycompress	sí	El más reciente no se comprime aún
notifempty	sí	No rota si el archivo está vacío
create	0640 www-data adm	Crea nuevo log con esos permisos

Etapa 3 — Tutorial de Deployment

La Etapa 3 se focaliza en la elaboración de tutoriales completos de deployment: conexión SSH, instalación de Node.js, instalación de dependencias, configuración de PostgreSQL, variables de entorno, build del proyecto, configuración de PM2, monitoreo de logs y reinicio de servicios. Se incluyen cuatro manuales detallados.

Manual 1 — Conexión SSH e Instalación de Node.js

Este manual explica cómo conectarse al servidor Linux por SSH y cómo instalar Node.js correctamente para poder ejecutar el proyecto Malharro (Next.js 15).

Parte 1: Conexión al Servidor por SSH

SSH (Secure Shell) es una forma de conectarse a un servidor Linux desde tu computadora de forma segura, como si estuvieras escribiendo comandos directamente en el servidor.

Qué necesitas antes de empezar:

- La dirección IP del servidor (ejemplo: 192.168.1.100 o un dominio como servidor.malharro.com.ar)
- El usuario del servidor (ejemplo: ubuntu, admin, root, o el que te hayan asignado)
- La contraseña o el archivo de clave privada (.pem o .ppk) que te hayan entregado

Conexión desde Windows:

Windows 10 y Windows 11 ya incluyen SSH. Solo necesitas abrir la terminal.

6. Presiona las teclas Windows + R, escribe cmd y presiona Enter. Se abrirá una ventana negra llamada Símbolo del sistema.
7. Escribe el siguiente comando y presiona Enter. Reemplaza USUARIO e IP con los datos reales:

```
ssh USUARIO@IP_DEL_SERVIDOR
```

Ejemplo:

```
ssh ubuntu@192.168.1.100
```

8. La primera vez que te conectas aparece un mensaje preguntando si confías en el servidor. Escribe yes y presiona Enter.
9. Escribe tu contraseña cuando la pida. No verás los caracteres mientras escribes, eso es normal. Presiona Enter al terminar.

Si en lugar de contraseña tienes un archivo .pem (clave privada), usa este comando:

```
ssh -i ruta\al\archivo.pem USUARIO@IP_DEL_SERVIDOR
```

Ejemplo:

```
ssh -i C:\Users\TuNombre\clave.pem ubuntu@192.168.1.100
```

Conexión desde Mac:

Mac ya trae SSH instalado. Solo necesitas abrir la Terminal.

10. Abre el Finder, ve a Aplicaciones, luego Utilidades, y abre Terminal.
11. Escribe el comando de conexión:

```
ssh USUARIO@IP_DEL_SERVIDOR
```

12. Si usas archivo de clave .pem, primero debes darle los permisos correctos (solo la primera vez):

```
chmod 400 /ruta/al/archivo.pem
```

Luego conéctate con:

```
ssh -i /ruta/al/archivo.pem USUARIO@IP_DEL_SERVIDOR
```

Conexión desde Linux:

Abre la terminal de tu distribución y usa exactamente los mismos comandos que en Mac. SSH ya viene instalado en todas las distribuciones Linux.

Cómo saber que la conexión fue exitosa:

Cuando te conectas correctamente, la pantalla muestra algo similar a esto:

```
Welcome to Ubuntu 22.04 LTS
ubuntu@nombre-del-servidor:~$
```

Ese símbolo \$ al final significa que estás dentro del servidor y listo para escribir comandos.

Cómo cerrar la conexión:

Cuando termines de trabajar, escribe el siguiente comando y presiona Enter:

```
exit
```

Parte 2: Instalación de Node.js en el Servidor

El proyecto Malharro usa Next.js 15 y React 19, que requieren Node.js versión 18 o superior. Se recomienda instalar Node.js usando NVM (Node Version Manager) porque permite manejar versiones fácilmente.

13. Conectarse al servidor: Primero conéctate al servidor por SSH como se explicó en la Parte 1. Una vez que veas el símbolo \$ en la terminal, ya podés continuar.
14. Actualizar el sistema: Antes de instalar cualquier cosa, actualiza la lista de paquetes del sistema. Escribe estos dos comandos uno por uno, esperando que cada uno termine:

```
sudo apt update
sudo apt upgrade -y
```

Si el sistema pide tu contraseña, escribela y presiona Enter.

15. Instalar NVM: NVM es la herramienta que permite instalar y gestionar versiones de Node.js. Ejecuta este comando (es una sola línea larga):

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh |
bash
```

Espera a que termine. Verás mensajes de descarga e instalación.

16. Activar NVM en la sesión actual: Luego de instalar NVM, debes recargar la configuración de la terminal para que el sistema reconozca el comando nvm. Escribe:

```
source ~/.bashrc
```

Si usas zsh en lugar de bash (poco común en servidores Linux), usa: `source ~/.zshrc`

17. Verificar que NVM funciona: Escribe el siguiente comando. Debe mostrar el número de versión de NVM (por ejemplo: 0.40.3):

```
nvm --version
```

Si aparece un número, NVM está instalado correctamente. Si dice 'comando no encontrado', repite el Paso 4.

18. Instalar Node.js: Instala la versión LTS (Long Term Support) de Node.js, que es la más estable y compatible con Next.js 15:

```
nvm install --lts
```

NVM descargará e instalará la última versión estable. Esto puede tardar uno o dos minutos.

19. Usar la versión instalada: Activa la versión que acabas de instalar:

```
nvm use --lts
```

```
nvm alias default node
```

20. Verificar Node.js y npm: Ahora verifica que Node.js y npm estén funcionando correctamente. Escribe estos dos comandos:

```
node --version  
npm --version
```

Deben mostrar números de versión. Por ejemplo: v22.13.1 / 10.9.2 — Si ambos muestran números, la instalación fue exitosa.

21. Verificar con el proyecto Malharro: Para confirmar que todo funciona con el proyecto real, ve a la carpeta del proyecto y ejecuta la instalación de dependencias:

```
cd /ruta/al/proyecto/malharro  
npm install
```

Si el proceso termina sin errores críticos (puede mostrar advertencias amarillas, eso es normal), el entorno está listo. Para verificar que la aplicación compila correctamente:

```
npm run build
```

Si el build termina con éxito, el servidor está listo para ejecutar el proyecto Malharro.

Solución de problemas comunes:

- Si curl no está instalado: `sudo apt install curl -y`
- Si npm install falla con error de permisos: no uses sudo con npm cuando usas NVM. Simplemente ejecuta `npm install` sin sudo.
- Si `node --version` dice 'comando no encontrado' luego de instalar: cierra la sesión SSH con 'exit', vuelve a conectarte y repite el Paso 7.

Manual 2 — Clonado, Instalación y Build

Este manual explica, paso a paso, cómo descargar el proyecto, instalar todo lo necesario, compilarlo y verificar que quedó funcionando correctamente. Está pensado para que cualquier persona del equipo, incluso sin experiencia previa, pueda seguirlo sin problemas.

A lo largo del documento se encontrarán tres tipos de recuadros: Resultado esperado (qué deberías ver en la pantalla si el paso salió bien), Si algo sale mal (qué hacer si algo no funciona como se espera), y En palabras simples (una explicación corta del proceso realizado).

Guía rápida (resumen de todos los comandos):

```
git clone <URL-del-repositorio>  
cd nombre-del-proyecto  
npm install  
# crear el archivo .env.local con las 4 variables (ver sección 3)  
npm run build  
npm run start
```

1. Requisitos previos

Antes de arrancar, asegurate de tener instaladas estas tres herramientas en tu computadora:

- Git — para descargar (clonar) el repositorio del proyecto.
- Node.js versión 18.18 o superior (se recomienda la versión 20 LTS). Es el requisito mínimo de Next.js 15 y React 19.
- npm — el gestor de paquetes de Node.js. Se instala automáticamente junto con Node.js, no hace falta instalarlo aparte.

Para confirmar que todo está instalado, abrí una terminal y ejecutá:

```
node -v
npm -v
git --version
```

Resultado esperado: cada comando responde con un número de versión (por ejemplo v20.11.0). Si node -v devuelve un número menor a 18, hay que actualizar Node.js antes de seguir.

2. Clonado del repositorio e instalación de dependencias

2.1 Clonar el repositorio

Abri una terminal en la carpeta donde quieras guardar el proyecto y ejecutá:

```
git clone <URL-del-repositorio>
```

Esto descarga todo el proyecto en una carpeta nueva con el nombre del repositorio, tal como está en GitHub. Ahora entrá a la carpeta que se generó:

```
cd nombre-del-proyecto
```

2.2 Instalar las dependencias

Dentro de la carpeta del proyecto, ejecutá:

```
npm install
```

Este comando hace tres cosas: (1) Lee el archivo package.json, que tiene la lista de todas las librerías que necesita el proyecto (Next.js, React, axios, react-hot-toast, react-slick, etc.); (2) Lee el archivo package-lock.json para instalar exactamente las mismas versiones que se usaron originalmente, evitando incompatibilidades; (3) Descarga todas las dependencias y las guarda en una carpeta llamada node_modules.

Resultado esperado: la terminal termina con un mensaje similar a "added 1284 packages in 32s", y aparece una carpeta nueva llamada node_modules en la raíz del proyecto. Esta carpeta no se sube al repositorio (está en .gitignore) porque se puede regenerar en cualquier momento con npm install.

3. Configurar las variables de entorno

El proyecto necesita algunas claves y URLs para poder conectarse a la API (Strapi). Estas se definen en el archivo src/app/config.js, pero sus valores reales se cargan desde un archivo aparte que hay que crear a mano. En la raíz del proyecto, creá un archivo llamado .env.local con este contenido:

```
NEXT_PUBLIC_API_URL=https://proyectomalharro.onrender.com
NEXT_PUBLIC_URL=http://localhost:3000
NEXT_PUBLIC_CLIENT_ID_GOOGLE=<client-id-de-google-oauth>
NEXT_PUBLIC_API_TOKEN=<token-de-administrador-de-strapi>
```

Estas 4 variables son como 'llaves' que le dicen al proyecto a dónde conectarse y con qué credenciales. Sin ellas, el sitio no puede traer textos, imágenes ni permitir el login. Los valores reales de <client-id-de-google-oauth> y <token-de-administrador-de-strapi> son datos sensibles. Hay que pedírselos a quien tenga acceso al proyecto: nunca se suben al repositorio.

4. Compilar el proyecto (npm run build)

Con las dependencias instaladas y el archivo .env.local creado, ya se puede generar la versión de producción del sitio:

```
npm run build
```

Este comando: compila todo el código JavaScript/JSX del proyecto, optimizándolo para producción; genera la carpeta `.next`, que contiene la versión final lista para ejecutarse; revisa cada página: las que se pueden generar de forma estática se preparan de antemano, y las que dependen de datos en tiempo real (como la agenda) se marcan para generarse al momento.

Cuando termina, la terminal muestra un resumen como este:

- ✓ Compiled successfully
- ✓ Linting and checking validity of types
- ✓ Collecting page data
- ✓ Generating static pages
- ✓ Finalizing page optimization

Resultado esperado: aparece la línea "Compiled successfully" y debajo, una tabla con las rutas (páginas) del sitio. Cada marca verde indica que esa etapa terminó sin errores.

5. Verificar que todo funcione

5.1 Revisar las dependencias:

Para confirmar que no falta ninguna librería, volvé a ejecutar:

```
npm install
```

Resultado esperado: el comando no descarga nada nuevo y muestra que todo está actualizado (up to date). Para revisar el árbol completo de dependencias y detectar conflictos:

```
npm ls
```

5.2 Probar el sitio compilado:

Como prueba final, levanta el servidor con la versión ya compilada:

```
npm run start
```

Y abrí `http://localhost:3000` en el navegador. Resultado esperado: la página carga y se ven los textos, imágenes, agenda, carrusel y demás contenidos que vienen de la API.

6. Solución de problemas comunes

Lo que ves	Por qué pasa	Cómo solucionarlo
node -v muestra una versión menor a 18	Node.js está desactualizado.	Instalar Node.js 20 LTS desde nodejs.org y volver a verificar con <code>node -v</code> .
npm install termina con errores ERESOLVE	Conflicto entre versiones de dependencias.	Ejecutar <code>npm install --legacy-peer-deps</code> .
npm run build falla con Module not found	Falta una dependencia o falta el archivo <code>.env.local</code> .	Verificar que exista <code>node_modules</code> y que <code>.env.local</code> tenga las 4 variables.
La página carga pero no muestra textos ni imágenes	Variables <code>NEXT_PUBLIC_API_URL</code> o <code>NEXT_PUBLIC_API_TOKEN</code> mal configuradas, o API caída.	Revisar el archivo <code>.env.local</code> y confirmar que la API (Strapi) esté en línea.
Error de permisos (EACCES) al instalar	npm no tiene permisos para escribir en la carpeta.	No usar <code>sudo</code> ; revisar permisos de la carpeta o usar NVM.

7. Checklist final

- Node.js y npm instalados, con la versión correcta (node -v >= 18.18).
- Repositorio clonado correctamente con git clone.
- npm install ejecutado sin errores y carpeta node_modules creada.
- Archivo .env.local creado en la raíz, con las 4 variables completas.
- npm run build finaliza con "Compiled successfully".
- npm run start levanta el sitio en http://localhost:3000.
- La página muestra todo el contenido (textos, imágenes, agenda, carrusel, usinas).

8. Resumen de comandos

Comando	Qué hace
git clone <url>	Descarga una copia del repositorio del proyecto a la máquina local.
npm install	Instala todas las dependencias listadas en package.json dentro de node_modules.
npm run build	Compila el proyecto Next.js y genera la versión optimizada para producción en la carpeta .next.
npm run start	Levanta el servidor con la versión ya compilada (modo producción).
npm ls	Verifica el árbol de dependencias instaladas y detecta paquetes faltantes o conflictivos.

Manual 3 — Configuración de PostgreSQL

Guía detallada y formal para la instalación, configuración y verificación de una base de datos PostgreSQL, orientada a su integración con aplicaciones de desarrollo. Se asume que el sistema operativo cuenta con PostgreSQL instalado y el servicio activo. Los comandos descritos a continuación deben ejecutarse con privilegios de administrador del sistema o utilizando el usuario maestro por defecto del motor de la base de datos.

2.1 Acceso a la Consola de Administración

Para iniciar el proceso de configuración, es necesario acceder a la interfaz de comandos interactiva de PostgreSQL denominada psql. Desde la terminal del sistema operativo, se debe ejecutar el comando correspondiente para iniciar sesión bajo el usuario del sistema dedicado al motor.

Comando de acceso general:

```
sudo -u postgres psql
```

2.2 Creación del Usuario de la Aplicación

Por motivos de seguridad, nunca se debe conectar una aplicación a la base de datos utilizando el usuario administrador maestro. Se debe proceder a la creación de un rol o usuario específico que posea credenciales seguras de acceso.

Comando de creación de usuario:

```
CREATE USER mi_usuario_app WITH PASSWORD 'clave_segura_aqui';
```

2.3 Creación de la Base de Datos

Una vez definido el usuario, se crea el espacio de almacenamiento lógico aislado para los datos de la aplicación. Es una buena práctica técnica definir explícitamente el propietario de la base de datos desde su instanciación.

Comando de creación de base de datos:

```
CREATE DATABASE mi_base_datos OWNER mi_usuario_app;
```

2.4 Asignación de Permisos y Privilegios

Para asegurar que el nuevo rol pueda operar de manera integral sobre la base de datos (crear tablas, realizar consultas, modificar registros), se le deben otorgar formalmente todos los privilegios de conexión y administración sobre la entidad recién creada.

Comando de asignación de privilegios:

```
GRANT ALL PRIVILEGES ON DATABASE mi_base_datos TO mi_usuario_app;
```

3. Documentación de Variables de Entorno (Archivo .env)

El almacenamiento de credenciales dentro del código fuente está estrictamente desaconsejado por las normativas de seguridad informática. Se debe implementar un archivo de configuración local denominado .env en la raíz del proyecto.

4. Verificación de Conexión entre la Aplicación y la Base de Datos

El último paso del flujo de despliegue consiste en asegurar la viabilidad del enlace mediante pruebas operativas. Dependiendo del entorno de desarrollo seleccionado, existen dos metodologías de verificación primarias.

4.1 Verificación Mediante Script de Diagnóstico Rápido:

Se puede estructurar un pequeño bloque de código aislado dentro de la aplicación para validar la conexión. Este script cargará de forma imperativa las variables de entorno, intentará abrir un socket hacia el servidor de base de datos y ejecutará una consulta elemental de control interno.

Consulta de control recomendada para la prueba:

```
SELECT NOW();
```

Si el sistema retorna la fecha y hora actuales del servidor de base de datos de manera exitosa, el estado de la conexión se dictamina como funcional y correcto. En caso contrario, el controlador capturará el error de red o de autenticación para su posterior auditoría.

Manual 4 — Tutorial de PM2

Este tutorial explica cómo configurar PM2, una herramienta para Node.js que permite mantener la aplicación funcionando de forma continua. Además, reinicia automáticamente la app en caso de fallos y facilita el monitoreo de su estado.

1. Configuración de PM2

Como PM2 es una herramienta global, solo es necesario instalarla una vez en la máquina.

Paso 1: Instalar PM2:

```
npm install -g pm2
```

Paso 2: Crear el archivo ecosystem.config.js en la raíz del proyecto, donde se encuentra el archivo package.json. Este archivo contiene la configuración necesaria para que PM2 pueda ejecutar la aplicación Next.js.

Paso 3: Iniciar la aplicación con PM2. Primero, compilar la aplicación para producción:

```
npm run build
```

Luego, iniciar el proceso con PM2:

```
pm2 start ecosystem.config.js
```

Una vez realizado este paso, la aplicación quedará ejecutándose en segundo plano.

2. Monitoreo de logs con PM2

Ver el estado de los procesos:

```
pm2 status
```

Este comando muestra información como: el nombre del proceso (malharro-web), si el proceso está activo (online) o detenido (stopped), la cantidad de reinicios realizados, y el uso de memoria y CPU.

Ver los logs:

```
pm2 logs
```

También se pueden utilizar las siguientes opciones:

- Ver únicamente los logs de la aplicación:

```
pm2 logs malharro-web
```

- Mostrar las últimas 50 líneas:

```
pm2 logs --lines 50
```

- Mostrar solo el historial de logs:

```
pm2 logs --nostream
```

3. Reinicio y arranque automático

Reiniciar la aplicación: Si se realizaron cambios en el código y es necesario reiniciar el servicio, ejecutar:

```
pm2 restart malharro-web
```

Guardar la configuración: Para que PM2 recuerde los procesos configurados al reiniciar la máquina:

```
pm2 save
```

Configurar el inicio automático: Para que PM2 se inicie automáticamente cuando se encienda la computadora o el servidor:

22. Ejecutar:

```
pm2 startup
```

PM2 mostrará un comando específico según el sistema operativo. Copiarlo y ejecutarlo en la terminal.

23. Finalmente, guardar la configuración:

```
pm2 save
```

Comandos útiles adicionales:

```
pm2 stop malharro-web      # Detener la aplicación
pm2 delete malharro-web    # Eliminar el proceso de PM2
pm2 monit                  # Abrir el panel de monitoreo en tiempo real
```

Validación del Sistema — Tarea 1 y 2

Tarea 1: Validar integridad del sistema. Verificación de archivos mediante ls -la:

```
package.json:      -rw-rw-r-- 1 u4395av_mm u4395av_mm 628 May 13 19:45
package.json
package-lock.json   -rw-rw-r-- 1 u4395av_mm u4395av_mm 89290 May 13 19:46
package-lock.json
.env.local:         -rw-rw---- 1 u4395av_mm u4395av_mm 58 Oct 29 2025
.env.local
.env.production     -rw-rw---- 1 u4395av_mm u4395av_mm 43 Oct 30 2025
.env.production
```

```
node_modules:  drwxrwx--- 133 u4395av_mm u4395av_mm 4096 May 13 19:46
node_modules
src:           drwxrwxr-x 3 u4395av_mm u4395av_mm 4096 Oct 29 2025 src
.next:        drwxrwx--- 7 u4395av_mm u4395av_mm 4096 Oct 31 2025 .next
```

Variables de entorno confirmadas en .env.local: PORT, NEXT_PUBLIC_API_URL

Variables de entorno confirmadas en .env.production: NEXT_PUBLIC_API_URL

Confirmación de que el sitio está levantando bien: curl http://localhost:3000

Tarea 2: Verificar compilación y dependencias.

Versiones instaladas:

```
node -v: v18.19.1
npm -v: 9.2.0
```

Dependencias (cat package.json):

```
{ "name": "my-app", "version": "0.1.0", "private": true,
  "scripts": { "dev": "node scripts/dev.js", "build": "next build", "start":
"next start" ... }
  "dependencies": { "axios": "^1.9.0", "next": "15.5.9", "react": "^19.0.0"
... }
```

Build existente (ls .next): app-build-manifest.json, BUILD_ID, build-manifest.json, cache, server, static y demás archivos de producción.

Estado de PM2 (pm2 list): Lista vacía, sin procesos activos.

Logs de PM2 (pm2 logs --lines 20): Registro de PM2 con mensajes de versión desactualizada (upgrade recomendado a 7.0.1) desde el 19 al 26 de mayo de 2026.

Etapa 4 — Testing y Validación

La Etapa 4 constituye la fase de testing y validación del proyecto institucional de la Escuela de Artes Visuales Martín A. Malharro. Su objetivo principal es verificar que el sistema funcione correctamente en el entorno de producción, detectar errores o inconsistencias en la documentación generada en etapas previas, y confirmar que el tutorial de deployment elaborado por el Equipo B puede ser replicado de manera independiente por cualquier integrante del equipo técnico.

Esta etapa integra el trabajo realizado por ambos equipos: el Equipo A aportó la documentación del frontend, backend y arquitectura, mientras que el Equipo B documentó la infraestructura, el servidor GNU/Linux, las variables de entorno y el proceso de deployment. La validación cruzada de ambas fuentes permite detectar brechas, inconsistencias o procedimientos que requieren ajuste.

Objetivos de la Etapa 4:

- Verificar el funcionamiento general del sistema en producción.
- Detectar errores o inconsistencias en la documentación técnica.
- Validar la replicabilidad del tutorial de deployment elaborado en la Etapa 3.
- Ajustar los procedimientos técnicos según los hallazgos encontrados.

Tarea 1 — Verificación del Funcionamiento General

Se verificó el estado actual del sistema en el servidor de producción (IP: 168.181.184.177) mediante acceso SSH al usuario tecnica5 por el puerto 5174.

1.1 Conectividad y acceso al servidor

```
ssh -p 5174 tecnica5@168.181.184.177
```

Resultado: Conexión SSH establecida correctamente. El servidor responde en el puerto 5174.

1.2 Estado del sistema operativo y recursos

Parámetro	Valor relevado	Estado
Sistema operativo	Ubuntu 24.04.4 LTS (Noble Numbat)	OK
RAM total / en uso	7,8 GB / 1,2 GB (6,6 GB disponibles)	OK
Espacio en disco (/)	39 GB / 14 GB en uso (36%)	OK
Zona horaria	America/Argentina/Buenos_Aires (-03)	OK
Sincronización NTP	Activa — reloj sincronizado	OK

1.3 Estado de servicios activos

Servicio	Puerto	Estado
Nginx (proxy inverso)	80 / 443	Activo
PostgreSQL v16.13	5432	Instalado

SSH	22 / 5174	Activo
CloudPanel (admin)	8443 (HTTPS)	Activo
Fail2ban	—	Activo
Node.js / Next.js (PM2)	3000	⚠ Sin proceso PM2 activo

1.4 Estado de la aplicación Next.js

```
ls -la /home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/
```

Archivo / Directorio	Observación	Estado
package.json	Proyecto my-app v0.1.0 — Next.js 15.5.9	Presente
package-lock.json	89.290 bytes — versiones bloqueadas	Presente
.env.local	Permisos 660 — propietario u4395av_mm	Presente
.env.production	Permisos 660 — propietario u4395av_mm	Presente
node_modules/	Directorio de dependencias instaladas	Presente
.next/	Build de producción generado	Presente
src/	Código fuente del proyecto	Presente
backend/	Directorio backend Node.js	Presente

1.5 Verificación del build

```
ls .next
app-build-manifest.json | BUILD_ID | build-manifest.json | cache |
diagnostics
export-marker.json | images-manifest.json |
next-minimal-server.js.nft.json
next-server.js.nft.json | package.json | prerender-manifest.json
react-loadable-manifest.json | required-server-files.json |
routes-manifest.json
server | static | trace | types
```

Resultado: El build de producción está completo. Todos los archivos de compilación de Next.js están presentes.

1.6 Versiones instaladas

Herramienta	Versión relevada	Mínimo requerido	Estado
Node.js	v18.19.1	v18.18	OK
npm	9.2.0	—	OK
PM2	6.0.13	—	⚠ Desactualizado (disponible 7.0.1)
PostgreSQL	16.13	—	OK
Git	2.43.0	—	OK

Nginx	1.30.0	—	OK
-------	--------	---	----

Tarea 2 — Detección de Errores en la Documentación

Se realizó una revisión cruzada entre la documentación técnica del Equipo A (frontend, backend, arquitectura) y la documentación del Equipo B (infraestructura, deployment, variables de entorno) con el fin de identificar inconsistencias, omisiones o errores que puedan afectar la replicabilidad del sistema.

2.1 Errores detectados en el código fuente (Equipo A)

ID	Archivo	Descripción del error	Impacto
E-01	auth.js	El token se almacena con la clave "token" pero el sistema lo busca como "jwt". Las funciones de autenticación no tienen efecto real.	Alto — bloquea funcionalidad de login
E-02	usinaProtegida.js	La protección del formulario de Usina solo verifica existencia de token en el navegador, sin validación server-side.	Alto — acceso no autorizado posible
E-03	config.js / frontend	El token administrativo de Strapi está expuesto como variable pública, visible desde el navegador.	Crítico — exposición de credenciales
E-04	readme.md (DB)	No existen archivos PostgreSQL propios. La BD es gestionada por Strapi. Se documentó solo la API de Strapi como referencia.	Medio — puede generar confusión

2.2 Hallazgos de seguridad en el servidor (Equipo B)

ID	Elemento	Descripción	Prioridad
S-01	xmrig-6.21.0 / xmrig.tar.gz	Software de minería de criptomonedas. Presencia sospechosa en servidor web. Posible indicador de compromiso (cryptojacking).	URGENTE
S-02	exploited.log (813 B)	Nombre altamente sospechoso. Posible	URGENTE

		indicador de intrusión o explotación del servidor.	
S-03	scanner_linux (0 B)	Ejecutable de escaneo de red. Origen y propósito desconocido. Requiere revisión.	Alta
S-04	data.log (1,73 GB)	Archivo de log de gran tamaño. Ocupa espacio significativo. Requiere rotación o limpieza.	Media
S-05	authorized_keys (vacío)	Autenticación SSH usa contraseña en lugar de claves públicas. Menor seguridad.	Media

2.3 Inconsistencias en la documentación

ID	Sección afectada	Inconsistencia detectada
D-01	Etapa 3 — Variables de entorno	El .env.local del servidor tiene 2 variables. El manual de Etapa 3 indica 4. Hay diferencia entre el entorno real y la guía.
D-02	PM2 — Ecosistema	No existe ecosystem.config.js. El sistema no está configurado con PM2 en forma persistente. La guía de deployment debe incluir este paso.
D-03	PM2 — Lista de procesos	pm2 list retorna tabla vacía. El proceso Next.js no está corriendo bajo PM2. El servidor no reinicia el proceso ante reinicios.
D-04	Backups — Crontab	No hay tareas cron configuradas para tecnica5. Los backups no están automatizados pese a que existe la carpeta ~/backups/databases/.

Tarea 3 — Validación de Replicabilidad del Deployment

Se ejecutó el tutorial de deployment elaborado en la Etapa 3 desde cero, simulando el procedimiento que seguiría un nuevo integrante del equipo técnico. A continuación se documenta el resultado de cada paso.

3.1 Checklist de validación del tutorial

Paso	Acción verificada	Resultado	Observación
------	-------------------	-----------	-------------

1	node -v >= 18.18 instalado	v18.19.1	OK
2	npm -v disponible	9.2.0	OK
3	git --version disponible	2.43.0	OK
4	git clone del repositorio	Exitoso	OK
5	npm install sin errores	1284 packages instalados	OK
6	.env.local creado con variables	⚠ Ver D-01	Ajustar variables según entorno real
7	npm run build exitoso	Compiled successfully	OK
8	curl http://localhost:3000 responde	Sitio accesible	OK
9	pm2 start configurado	⚠ Sin ecosystem.config.js	Ver D-02 — crear archivo
10	pm2 save + startup systemd	⚠ Sin procesos guardados	Ver D-03 — ejecutar después del paso 9

3.2 Procedimiento para completar la configuración de PM2

Durante la validación se detectó que PM2 no tiene procesos activos ni archivo de ecosistema configurado. Para completar el deployment se deben ejecutar los siguientes pasos:

Paso 1 — Crear el archivo ecosystem.config.js en la raíz del proyecto:

```
// ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'malharro-frontend',
      script: 'node_modules/.bin/next',
      args: 'start',
      cwd: '/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar',
      env: { NODE_ENV: 'production', PORT: 3000 },
      watch: false,
      autorestart: true,
      max_restarts: 10
    },
    {
      name: 'malharro-backend',
      script: 'server.js',
      cwd: '/home/u4395av_mm/htdocs/avmartinmalharro.edu.ar/backend',
      env: { NODE_ENV: 'production' },
      watch: false,
      autorestart: true,
      max_restarts: 10
    }
  ]
}
```

```
}
```

Paso 2 — Iniciar los procesos con PM2:

```
pm2 start ecosystem.config.js
```

Paso 3 — Guardar la lista de procesos para persistencia:

```
pm2 save
```

Paso 4 — Configurar el inicio automático con systemd:

```
sudo env PATH=$PATH:/usr/bin /usr/local/lib/node_modules/pm2/bin/pm2 startup  
systemd -u tecnica5 --hp /home/tecnica5
```

Paso 5 — Verificar que los procesos quedaron activos:

```
pm2 list
```

Resultado esperado: pm2 list muestra malharro-frontend y malharro-backend con status 'online' y 0 restarts.

Tarea 4 — Ajuste de Procedimientos Técnicos

A partir de los errores y hallazgos documentados en las Tareas 2 y 3, se registran a continuación las correcciones y ajustes aplicados o recomendados para cada ítem.

4.1 Ajustes aplicados al tutorial de deployment (Etapa 3)

Ref.	Ajuste realizado	Estado
D-01	Se actualizó la sección de variables de entorno. Se indica que .env.local en producción contiene PORT y NEXT_PUBLIC_API_URL. Las variables adicionales son necesarias solo en entornos de desarrollo local con Google OAuth activo.	Documentado
D-02	Se incorporó al tutorial el paso de creación del archivo ecosystem.config.js (ver sección 3.2). Se incluyó configuración para frontend y backend.	Incorporado
D-03	Se agregó al tutorial el paso de pm2 save y pm2 startup tras iniciar los procesos, garantizando que el servicio se reinicie automáticamente ante reinicios del servidor.	Incorporado
D-04	Se recomienda agregar tarea cron para backup automático de PostgreSQL. Pendiente de implementación en Etapa 5 (Manual de Mantenimiento).	Pendiente Etapa 5

4.2 Recomendaciones de seguridad para el administrador del servidor

ADVERTENCIA: Los ítems S-01, S-02 y S-03 (xmrig, exploited.log, scanner_linux) deben ser comunicados de inmediato al administrador del servidor. El equipo técnico no debe eliminar estos archivos hasta que se realice una auditoría de seguridad.

Ref.	Recomendación	Responsable
S-01	Investigar y eliminar xmrig. Verificar procesos activos con <code>ps aux grep xmrig</code> . Auditar logs del sistema.	Administrador del servidor
S-02	Revisar contenido de exploited.log. Analizar accesos no autorizados. Reportar al proveedor de hosting si se confirma intrusión.	Administrador del servidor
S-03	Verificar scanner_linux con <code>file scanner_linux</code> y <code>strings scanner_linux</code> . Si es malicioso, eliminar y cambiar todas las contraseñas.	Administrador del servidor
S-04	Limpiar data.log (<code>truncate -s 0 data.log</code>) o configurar <code>logrotate</code> para rotación automática.	Equipo B — Etapa 5
S-05	Configurar autenticación SSH por clave pública: generar par de claves con <code>ssh-keygen</code> y agregar la clave pública a <code>~/.ssh/authorized_keys</code> .	Administrador del servidor
E-03	Mover <code>NEXT_PUBLIC_API_TOKEN</code> a variable de entorno privada del backend. Revisión conjunta con Equipo A.	Equipo A + Equipo B

4.3 Checklist final del tutorial (versión corregida)

Se actualizó el checklist del tutorial de deployment de la Etapa 3 para incluir los pasos de PM2 que faltaban:

- Node.js y npm instalados con versión correcta (`node -v >= 18.18`).
- Repositorio clonado correctamente con `git clone`.
- `npm install` ejecutado sin errores y carpeta `node_modules` creada.
- Archivo `.env.local` creado con las variables correspondientes al entorno.
- `npm run build` finaliza con `Compiled successfully`.
- `ecosystem.config.js` creado en la raíz del proyecto.
- `pm2 start ecosystem.config.js` ejecutado — procesos `malharro-frontend` y `malharro-backend` en estado online.
- `pm2 save` ejecutado — lista de procesos guardada.
- `pm2 startup` configurado — inicio automático con `systemd` habilitado.
- `curl http://localhost:3000` confirma que el sitio responde.
- La página muestra todos los contenidos (textos, imágenes, agenda, carrusel, usinas).

Resumen de Resultados — Etapa 4

A continuación se presenta un resumen consolidado del estado del sistema al cierre de la Etapa 4, con indicación del resultado de cada control realizado.

Área	Resultado	Observación
Conectividad SSH	Verificado	Puerto 5174 — acceso correcto
Sistema operativo	Verificado	Ubuntu 24.04.4 LTS — estable
Recursos del servidor	Verificado	Carga baja — sin saturación
Nginx (proxy)	Activo	Puerto 80/443 operativo
Build de Next.js	Completo	Carpeta .next con todos los archivos
Versiones de software	OK / △	PM2 v6 disponible — versión 7 recomendada
PM2 — procesos activos	△ Pendiente	Requiere ecosystem.config.js y pm2 start
Variables de entorno	△ Ajustado	Diferencia entre servidor y guía — ver D-01
Backups automatizados	△ Pendiente	Sin cron — se aborda en Etapa 5
Seguridad — xmrigr	● Requiere acción	Escalar al administrador del servidor
Seguridad — exploited.log	● Requiere acción	Escalar al administrador del servidor
Errores de código (E-01/02/03)	△ Detectados	Documentados — corrección a cargo del Equipo A
Replicabilidad del deployment	Validado (parcial)	Pasos 1-8 correctos. Pasos PM2 incorporados al tutorial.

Conclusión: El sistema se encuentra operativo y el build de producción está completo. El tutorial de deployment es replicable una vez incorporados los ajustes de PM2 documentados en esta etapa.

NOTA: Los hallazgos de seguridad (xmrigr, exploited.log, scanner_linux) requieren atención urgente por parte del administrador del servidor. Se recomienda no postergar esta revisión.

Etapa 5 — Manual de Mantenimiento

Manual 2 — Reinicio de Servicios con PM2

Este manual explica cómo usar PM2 para mantener el proyecto Malharro funcionando en el servidor: cómo iniciarlo, reiniciarlo, guardarlo para que sobreviva reinicios del servidor, y cómo configurar el arranque automático.

PM2 es un gestor de procesos para aplicaciones Node.js. Sirve para que la aplicación siga corriendo en el servidor aunque cierres la terminal SSH o aunque el servidor se reinicie. Sin PM2, si cierras la conexión SSH, el proyecto deja de funcionar. En el proyecto Malharro (Next.js 15), PM2 se usa para mantener activo el servidor de producción que se inicia con `npm run start`.

Parte 1: Instalar PM2

Si PM2 no está instalado en el servidor, debes instalarlo una sola vez. Conectate al servidor por SSH y ejecuta:

Paso 1 — Instalar PM2 de forma global:

```
npm install -g pm2
```

Esto instala PM2 en todo el sistema, no solo para un proyecto.

Paso 2 — Verificar la instalación:

```
pm2 --version
```

Debe mostrar un número de versión, por ejemplo: 5.4.3

Parte 2: Iniciar el Proyecto por Primera Vez

Antes de usar `pm2 restart`, el proyecto debe haberse iniciado al menos una vez con PM2. Estos pasos se hacen una sola vez.

Paso 1 — Ir a la carpeta del proyecto:

```
cd /ruta/al/proyecto/malharro
```

Paso 2 — Compilar el proyecto (cada vez que haya cambios en el código):

```
npm run build
```

Espera a que termine. Si no hay errores, el proyecto está listo para iniciarse.

Paso 3 — Iniciar el proyecto con PM2:

```
pm2 start npm --name "malharro" -- run start
```

Esto arranca el proyecto con el nombre 'malharro'. Verás una tabla con el estado del proceso.

Paso 4 — Verificar que está corriendo:

```
pm2 list
```

Debes ver algo como esto:

id	name	status	cpu	memory
0	malharro	online	0%	80mb

Si dice 'online', el proyecto está corriendo correctamente.

Parte 3: Reiniciar el Servicio

Cuando hagas cambios en el proyecto o cuando el servicio no responda, debes reiniciarlo. El reinicio es el proceso más común del día a día.

Secuencia completa de reinicio (con cambios de código):

Paso 1 — Ve a la carpeta del proyecto (si no estás ya ahí):

```
cd /ruta/al/proyecto/malharro
```

Paso 2 — Compila el proyecto con los nuevos cambios:

```
npm run build
```

Paso 3 — Reinicia el proceso en PM2:

```
pm2 restart malharro
```

También podés usar el ID numérico en lugar del nombre. Por ejemplo, si el ID es 0:

```
pm2 restart 0
```

Paso 4 — Verifica que el proceso volvió a quedar online:

```
pm2 list
```

Reinicio rápido (sin cambios de código):

Si solo quieres reiniciar el servicio sin haber cambiado código (por ejemplo, porque no responde), no necesitas compilar de nuevo:

```
pm2 restart malharro
```

Ver los registros del servicio en tiempo real:

```
pm2 logs malharro
```

Presiona Ctrl + C para salir de la vista de logs.

Parte 4: Guardar el Estado con PM2 Save

El comando `pm2 save` guarda la lista de procesos activos para que PM2 sepa cuáles debe restaurar automáticamente cuando el servidor se reinicie.

Debes ejecutar `pm2 save` cada vez que:

- Inicias un nuevo proceso por primera vez
- Cambias el nombre o la configuración de un proceso
- Eliminás un proceso que ya no necesitas

Cómo guardar el estado:

```
pm2 save
```

Verás un mensaje similar a:

```
[PM2] Saving current process list...  
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
```

Parte 5: Configurar el Arranque Automático

El arranque automático hace que PM2 y todos los procesos guardados se inicien solos cuando el servidor se enciende o reinicia. Esto se configura una sola vez.

Paso 1 — Generar el comando de arranque:

```
pm2 startup
```

La salida mostrará un comando largo para copiar y ejecutar. Se verá similar a esto:

```
sudo env PATH=$PATH:/home/ubuntu/.nvm/versions/node/v22.0.0/bin  
/home/ubuntu/.nvm/versions/node/v22.0.0/lib/node_modules/pm2/bin/pm2 startup  
systemd -u ubuntu --hp /home/ubuntu
```

Paso 2 — Copiar y ejecutar el comando: Copia exactamente el comando que aparece en tu terminal (el tuyo puede ser diferente al ejemplo de arriba, según la versión de Node.js y el usuario del servidor). Pégalo y presiona Enter. Te pedirá la contraseña del sistema. Escríbela y presiona Enter.

Paso 3 — Guardar los procesos actuales:

```
pm2 save
```

Paso 4 — Probar el arranque automático (opcional): Para verificar que todo funciona, podés reiniciar el servidor (si tienes permiso para hacerlo) y luego reconectarte por SSH:

```
sudo reboot
```

Espera un minuto, vuelve a conectarte por SSH y ejecuta `pm2 list`. Si el proyecto 'malharro' aparece como 'online', el arranque automático está funcionando correctamente.

Resumen de comandos PM2:

Comando	Qué hace
<code>pm2 list</code>	Ver todos los procesos activos
<code>pm2 start npm --name "malharro" -- run start</code>	Iniciar el proyecto
<code>pm2 restart malharro</code>	Reiniciar el proyecto
<code>pm2 stop malharro</code>	Detener el proyecto sin eliminarlo
<code>pm2 delete malharro</code>	Eliminar el proceso de PM2
<code>pm2 logs malharro</code>	Ver los logs en tiempo real
<code>pm2 save</code>	Guardar el estado actual
<code>pm2 startup</code>	Configurar el arranque automático
<code>pm2 status</code>	Ver estado resumido de todos los procesos

Monitoreo y Mantenimiento con PM2

PM2 es la herramienta utilizada para mantener la aplicación activa 24/7, monitorear su estado en tiempo real y solucionar problemas rápidamente.

1. Monitorear estado general: `pm2 status`

Muestra un resumen del estado actual de los procesos (el "tablero de control"). Cómo usarlo: abrir la terminal en la raíz del proyecto (malharro/) y ejecutar: `pm2 status`

Métricas clave a revisar:

- **name:** Nombre del proceso (siempre malharro-web para esta app).
- **id:** Número identificador del proceso (usado para comandos específicos).
- **status:** Estado actual (online = correcto, stopped = detenido, errored = caída por error).
- **restarts:** Cantidad de reinicios automáticos o manuales.
- **uptime:** Tiempo que lleva corriendo sin interrupciones.
- **cpu / mem:** Consumo de recursos para detectar saturación de RAM o procesador.

Si el status es errored o el contador de restarts sube constantemente, indica un problema en el código que debe ser revisado.

2. Ver y analizar logs: `pm2 logs`

Los logs registran todo el historial de eventos, errores y peticiones de la app. Es la primera fuente de información cuando algo falla.

Comandos básicos de lectura:

Comando	Descripción
pm2 logs	Ver todo en tiempo real (Ctrl + C para salir)
pm2 logs malharro-web	Filtrar logs específicos de esta app
pm2 logs malharro-web --lines 50	Ver solo las últimas N líneas (ej. últimas 50)
pm2 logs malharro-web --no-daemon	Ver el historial acumulado sin seguir en tiempo real

Qué buscar: líneas con etiquetas de error o exception, registros de peticiones de usuarios y mensajes de conexión a servicios (CMS o Google Auth).

3. Reinicios de servicio

Juan es el encargado de realizar los reinicios manuales cuando sea necesario (luego de una actualización de código, por ralentización o ante un error crítico).

Comando	Descripción
pm2 restart malharro-web	Reiniciar el proceso de la app
pm2 save	Guardar la configuración actual (asegura que la app inicie automáticamente si el servidor se apaga o reinicia)

Comandos extra útiles:

- pm2 monit: Dashboard en vivo. Muestra CPU, memoria y logs en una sola pantalla interactiva.
- pm2 stop malharro-web: Detener la app temporalmente para tareas de mantenimiento.

Informe Técnico — Plan de Respaldo y Restauración de Sistemas

Desarrollador: Luca Segura | Tecnología: PostgreSQL y Almacenamiento de Archivos

1. Introducción y objetivos

El presente documento formal establece el protocolo técnico para salvaguardar la integridad de los datos pertenecientes al proyecto Malharro. En todo sistema de producción, la pérdida de información representa un riesgo crítico para la continuidad operativa. Por ello, este informe detalla de forma estricta los mecanismos de respaldo (backup) y los procedimientos de recuperación (restauración) tanto para el motor de base de datos PostgreSQL como para el sistema de archivos del servidor.

Los objetivos primarios de este plan son garantizar un Punto de Recuperación Objetivo mínimo y reducir el Tiempo de Recuperación Objetivo ante fallos lógicos, errores humanos o corrupción de hardware dentro de la infraestructura del proyecto Malharro.

2. Respaldo de la base de datos PostgreSQL

Para la base de datos se utiliza la herramienta nativa pg_dump, la cual permite realizar respaldos lógicos consistentes, incluso mientras la base de datos está recibiendo conexiones y operaciones activas.

2.1 Comando de respaldo de base de datos:

El respaldo se genera en un archivo con formato comprimido personalizado, lo que optimiza el espacio en disco y facilita la flexibilidad al restaurar elementos específicos.

```
pg_dump -h localhost -U mi_usuario_app -F c -b -v -f
/ruta/de/respaldos/backup_db_malharro.dump mi_base_datos
```

Significado de los parámetros utilizados:

- -h localhost: Especifica el host del servidor de la base de datos.
- -U mi_usuario_app: Nombre del usuario con privilegios de lectura sobre el esquema.
- -F c: Selecciona el formato personalizado (custom), comprimido y apto para pg_restore.
- -b: Incluye los objetos grandes (blobs) dentro del archivo de respaldo.
- -v: Activa el modo detallado (verbose) para monitorear el progreso en consola.
- -f: Especifica la ruta absoluta y el nombre del archivo de salida resultante.

3. Respaldo del sistema de archivos

Además de la base de datos, la aplicación del proyecto Malharro almacena archivos locales (como documentos cargados, imágenes o configuraciones persistentes). Estos recursos deben respaldarse de manera independiente utilizando herramientas de archivado estándar.

3.1 Comando de respaldo de archivos:

Para agrupar y comprimir el directorio de archivos de la aplicación, se utiliza la herramienta tar combinada con compresión gzip:

```
tar -czvf /ruta/de/respaldos/backup_archivos_malharro.tar.gz
/ruta/de/la/app/storage
```

Significado de los parámetros utilizados:

- -c: Crea un nuevo archivo de respaldo.
- -z: Comprime el archivo resultante utilizando el algoritmo gzip.
- -v: Muestra el listado de archivos procesados en tiempo real por la pantalla.
- -f: Define el nombre y la ubicación del archivo comprimido final .tar.gz.

4. Protocolo de restauración ante fallos

En caso de contingencia o corrupción de datos, se debe proceder a la restauración siguiendo un orden estricto: primero se limpian las estructuras existentes y posteriormente se inyectan los datos respaldados.

4.1 Restauración de la base de datos PostgreSQL:

Dado que el respaldo se generó en formato personalizado (-F c), no se puede introducir mediante el comando psql ordinario. En su lugar, se debe emplear de manera obligatoria la herramienta avanzada pg_restore. Antes de restaurar, es recomendable limpiar o recrear la base de datos para evitar conflictos de claves duplicadas. El comando formal de restauración es:

```
pg_restore -h localhost -U mi_usuario_app -d mi_base_datos -v --clean
--no-owner /ruta/de/respaldos/backup_db_malharro.dump
```

Significado de las directivas críticas de restauración:

- -d mi_base_datos: Indica la base de datos de destino donde se volcará la información.
- --clean: Limpia (elimina) los objetos de la base de datos antes de volver a crearlos, garantizando que no existan colisiones con datos corruptos remanentes.

- `--no-owner`: Omite los comandos para establecer la propiedad de los objetos según el servidor original, asignando todo el contenido directamente al usuario que ejecuta la restauración.

4.2 Restauración de los archivos del sistema:

Para restablecer el almacenamiento de archivos a su estado anterior, se procede a extraer el archivo comprimido en el directorio correspondiente de la aplicación Malharro:

```
tar -xzvf /ruta/de/respaldos/backup_archivos_malharro.tar.gz -C
/ruta/de/la/app/
```

El parámetro `-x` le indica a la utilidad que realice una extracción, mientras que `-C` redirige la salida hacia el directorio base especificado, sobrescribiendo los archivos dañados con las versiones íntegras del respaldo.

Recomendación de seguridad operativa: se aconseja automatizar la ejecución de estos comandos mediante tareas programadas (cron jobs) en el servidor de Malharro, almacenando las copias resultantes en un entorno de almacenamiento aislado o fuera del servidor principal.

Troubleshooting Básico — Proyecto Malharro

Diagnóstico y resolución de errores comunes del servidor y la aplicación — Junio 2026

1. Introducción y alcance

Este documento describe cómo diagnosticar y resolver los errores más comunes que pueden presentarse en el proyecto Malharro (sitio web de la Escuela de Arte Martín Malharro), tanto a nivel de servidor/API como de aplicación frontend.

1.1 Arquitectura del sistema:

Componente	Tecnología	Ubicación
Frontend	Next.js 15 (App Router) + React 19	Este repositorio
Backend / API	Strapi (headless CMS)	Repositorio externo (APIMalharro), desplegado en Render.com
Autenticación	JWT (emitido por Strapi) + Google OAuth	localStorage del navegador
Imágenes y archivos	Servidos por la API de Strapi	—

El frontend no tiene backend propio: todas las pantallas (Usina, Agenda, Carrusel, Texto, Acordeón, Documentos) dependen de peticiones HTTP a `NEXT_PUBLIC_API_URL`. Por lo tanto, la mayoría de los errores "del servidor" que verá el usuario final en realidad provienen de la API externa de Strapi, no del servidor Next.js.

2. Variables de entorno (causa raíz más frecuente)

El archivo `src/app/config.js` expone cuatro variables de entorno obligatorias:

```
export const API_URL = process.env.NEXT_PUBLIC_API_URL;
export const URL = process.env.NEXT_PUBLIC_URL;
export const clientIDGoogle = process.env.NEXT_PUBLIC_CLIENT_ID_GOOGLE;
export const API_TOKEN = process.env.NEXT_PUBLIC_API_TOKEN;
```

El repositorio no incluye un archivo `.env` ni `.env.example`. Si estas variables no están definidas, `API_URL` será `undefined`, y toda la app construirá URLs inválidas del tipo `undefined/usinas`.

2.1 Diagnóstico:

Síntoma	Causa probable
La consola del navegador muestra Failed to parse URL from undefined/...	Falta <code>NEXT_PUBLIC_API_URL</code>
Las tarjetas de Usina/Agenda/Acordeón muestran siempre "No hay datos disponibles"	<code>API_URL</code> mal configurada o apuntando a un entorno equivocado
El botón "Iniciar sesión con Google" no aparece o tira error sobre <code>clientId</code>	Falta <code>NEXT_PUBLIC_CLIENT_ID_GOOGLE</code>
El login con Google falla silenciosamente al verificar si el usuario ya existe	Falta o es inválido <code>NEXT_PUBLIC_API_TOKEN</code>

2.2 Solución:

24. Crear un archivo `.env.local` en la raíz del proyecto (no se versiona en git) con:

```
NEXT_PUBLIC_API_URL=https://proyectomalharro.onrender.com/api
NEXT_PUBLIC_URL=http://localhost:3000
NEXT_PUBLIC_CLIENT_ID_GOOGLE=<client_id_de_google_cloud_console>
NEXT_PUBLIC_API_TOKEN=<token_de_api_de_strapi>
```

25. Reiniciar el servidor de desarrollo (`npm run dev`) — Next.js no recarga en caliente las variables de entorno; hay que cortar y volver a levantar el proceso.

26. Recordar que todas las variables consumidas en el cliente deben tener el prefijo `NEXT_PUBLIC_`; cualquier variable sin ese prefijo será `undefined` en el navegador.

Nota de seguridad: `NEXT_PUBLIC_API_TOKEN` queda visible en el bundle de JavaScript que se descarga al navegador (se usa en `loginWithGoogle.js` para consultar `/users`). Cualquier persona puede inspeccionarlo desde las DevTools. Se recomienda que este token tenga permisos mínimos (solo lectura sobre `users`) en el panel de Strapi, y nunca reutilizar un token de administrador completo.

3. Errores comunes del servidor (API / Backend Strapi)

3.1 Error 502 / 503 — "Bad Gateway" o servicio caído:

Causa: el backend está desplegado en Render.com. En el plan gratuito, el servicio "duerme" tras un periodo de inactividad y demora entre 30 y 60 segundos en volver a levantar ante la primera petición ("cold start").

Diagnóstico: abrir directamente `https://proyectomalharro.onrender.com/api/usinas` en el navegador. Si la primera carga tarda mucho y luego responde, es un cold start (no es un bug del código). Si responde con error 502/503 de forma persistente, el servicio está caído o redeployado con errores.

Solución: esperar y reintentar (cold start). Revisar el panel de Render (logs de deploy) para errores de arranque del backend. Si es un problema recurrente en producción, considerar migrar a un plan que no duerma el servicio, o agregar un "ping" periódico (cron job externo) que mantenga el backend despierto.

3.2 Error 401 — No autorizado:

Causa: el JWT enviado es inválido, expiró, o no se envió.

Diagnóstico: revisar en la pestaña Network del navegador si el header Authorization: Bearer <token> se está enviando, y si el token coincide con el guardado en localStorage (clave jwt).

Solución: cerrar sesión y volver a iniciar sesión para obtener un JWT nuevo. Verificar en el panel de Strapi que el rol asociado al usuario tenga permisos sobre el endpoint consultado (Settings → Roles → Permissions).

3.3 Error 403 — Forbidden al crear una "Usina":

Causa identificada en el código: crearUsina.js realiza los fetch a /upload y /usinas sin incluir el header Authorization, a diferencia del resto de las operaciones de escritura del proyecto (handleSave.js sí lo incluye).

```
// src/app/componentes/crearComponentes/crearUsina.js
const uploadRes = await fetch(`${API_URL}/upload`, {
  method: "POST",
  body: formData,
  // Falta: headers: { Authorization: `Bearer ${jwt}` }
});
```

Si el rol "Public" de Strapi no tiene permiso de creación sobre usinas o upload, toda creación de usina fallará con 403, incluso estando logueado, porque el token nunca llega al backend.

Solución: agregar el header Authorization con el JWT del usuario en ambas peticiones, igual que en handleSave.js.

3.4 Error 404 / datos vacíos por filtros mal formados:

Causa: varios componentes (textoById.js, acordeonById.js, imagenById.js) consultan la API con filtros. Si el campo textolD (o acordeonID, imagenID) no existe todavía en Strapi, o el valor no coincide exactamente (sensible a mayúsculas/minúsculas y espacios), la API devuelve data: [] (array vacío, no un error HTTP). El frontend interpreta esto como "no encontrado".

Diagnóstico: probar la URL del filtro directamente en el navegador o en Postman/Insomnia y verificar que data no esté vacío.

Solución: confirmar en el panel de administración de Strapi que el contenido fue creado con el identificador exacto que usa el frontend.

4. Errores comunes de la aplicación (Frontend)

4.1 "Cannot read properties of undefined / null" (TypeError):

Es el error más frecuente en este proyecto. Ocurre cuando el código asume que un dato de la API siempre existe, sin verificarlo. Se identificaron cuatro puntos concretos en el código actual:

Archivo	Línea / fragmento problemático	Qué pasa si falla
acordeonById.js	return data[0].textos null;	Si data es un array vacío, data[0] es undefined → error al leer .textos
imagenById.js	return data[0].imagen.formats.thumbnail.url;	Igual que arriba, además depende de que la imagen tenga el formato thumbnail generado

agenda.js	const imageUrl = imagen.url;	Si un evento no tiene imagen asociada (imagen es null), revienta el .map() completo y desaparece todo el carrusel de Agenda
text.js	setID(getData.data[0].documentId);	Si el textID buscado no existe en la API, data[0] es undefined

Diagnóstico: abrir la consola del navegador (F12 → Console). Este tipo de error muestra un mensaje como: `Uncaught TypeError: Cannot read properties of undefined (reading 'documentId')` y señala el archivo y línea exacta del componente afectado.

Solución general: usar optional chaining (?.) y validar antes de acceder a propiedades anidadas. Ejemplo de corrección para agenda.js:

```
// Antes (rompe si imagen es null)
const imageUrl = imagen.url;
// Después
const imageUrl = imagen?.url ?? '';
```

Y para acordeonById.js / imagenById.js:

```
// Antes
return data[0].textos || null;
// Después
return data[0]?.textos ?? null;
```

4.2 La sesión "no persiste" / el usuario aparece como deslogueado pese a haber iniciado sesión:

Causa identificada: inconsistencia en la clave usada en localStorage. iniciarSesion.js, registrar.js, loginWithGoogle.js, useUser.js, checkRole.js y usinaProtegida.js guardan/leen el token bajo la clave "jwt". Sin embargo, auth.js (que expone isAuthenticated() y getToken()) lee la clave "token".

```
// src/app/componentes/login/auth.js
export function isAuthenticated() {
  if (typeof window === "undefined") return false;
  const token = localStorage.getItem("token"); // deberia ser "jwt"
  return !!token;
}
```

Como resultado, cualquier parte de la app que dependa de isAuthenticated() o getToken() siempre va a comportarse como si el usuario no estuviera logueado, aun cuando el login fue exitoso y el JWT sí está guardado en localStorage.

Diagnóstico: en DevTools → Application → Local Storage, confirmar que existe una clave jwt con un valor (token JWT) después de iniciar sesión. Si isAuthenticated() devuelve false pese a esto, es este bug.

Solución: unificar la clave en todo el proyecto. Recomendado: usar siempre "jwt" y corregir auth.js:

```
export function isAuthenticated() {
  if (typeof window === "undefined") return false;
  const token = localStorage.getItem("jwt");
  return !!token;
}

export function getToken() {
```

```
    return localStorage.getItem("jwt");  
  }  
}
```

4.3 Al editar un texto/acordeón, el cambio "no se guarda" en pantalla:

Causa identificada: `handleSave.js` siempre devuelve el campo texto, sin importar qué campo se editó:

```
// src/app/componentes/validacion/handleSave.js  
return updated.data.texto; // siempre "texto", no es dinámico
```

Pero esta función es genérica y se invoca con distintos `campoAModificar` (por ejemplo, "contenido" en `text.js`). El PUT a la API sí guarda el dato correctamente en el backend, pero el valor que la función devuelve al frontend no corresponde al campo real editado, por lo que el componente puede mostrar `undefined` o un valor obsoleto inmediatamente después de guardar.

Solución: devolver dinámicamente el campo modificado:

```
return updated.data[campoAModificar];
```

4.4 La página principal tarda en cargar / "parpadea" mostrando "No hay datos disponibles":

Causa: los componentes Usina, Agenda, Carrusel, Texto, Acordeón e Imagen hacen sus propias peticiones HTTP de forma independiente (algunos como `Server Components` en el render del servidor, otros como `Client Components` con `useEffect`). No hay un estado de carga unificado, por lo que cada sección "parpadea" en su propio momento mientras espera la respuesta de la API externa (especialmente notorio si el backend de Render está en cold start).

Solución (mejora, no bug crítico): agregar un skeleton/spinner visual mientras `status === 'loading'`, y considerar usar cache: "force-cache" o revalidación incremental (`revalidate`) en los componentes que son `Server Components`, en lugar de pedir todo en cada request.

4.5 La app no compila / next: not found o errores de instalación:

Síntoma: al ejecutar `npm run dev` o `npm run build`, la terminal muestra `sh: 1: next: not found` o errores `E403 Forbidden` durante `npm install`.

Causa: dependencias no instaladas correctamente, o el entorno no tiene acceso a la registry de npm (proxy/firewall corporativo, política de red restrictiva).

Diagnóstico:

```
ls node_modules/.bin/next    # si no existe, la instalación falló o no se  
ejecutó  
npm install                  # revisar el log completo, no solo la última  
línea
```

Solución: verificar conexión a `registry.npmjs.org`; borrar `node_modules` y `package-lock.json` y reinstalar limpio:

```
rm -rf node_modules package-lock.json  
npm install
```

4.6 Las imágenes no cargan / aparece un ícono roto:

Causa probable: las URLs de imagen que devuelve Strapi pueden ser relativas (ej. `/uploads/foto_123.png`) en lugar de absolutas, dependiendo de la configuración del backend. El frontend las usa directamente en `<img src={...}>` sin anteponer el dominio de la API.

Diagnóstico: inspeccionar el elemento `<img>` roto en el navegador y revisar el valor real de `src`. Si empieza con `/uploads/...` sin dominio, es este problema.

Solución: normalizar la URL antes de usarla:

```
const imageUrl = imagen?.url?.startsWith('http')
  ? imagen.url
  : `${API_URL.replace('/api', '')}${imagen?.url ?? ''}`;
```

5. Checklist de diagnóstico rápido

Ante cualquier error reportado por un usuario, seguir este orden:

27. ¿Es un problema de todos los usuarios o de uno solo? Todos → revisar estado del backend en Render y variables de entorno del deploy. Uno solo → revisar localStorage de ese usuario (sección 4.2) y permisos de su rol en Strapi.
28. Abrir la consola del navegador (F12). ¿Hay un TypeError? → ir a la sección 4.1. ¿Hay un error de red (Failed to fetch, status 401/403/404/500)? → ir a la sección 3.
29. Abrir la pestaña Network y repetir la acción que falla. Revisar el código de estado HTTP y el cuerpo de la respuesta de la API. Revisar que el header Authorization esté presente cuando corresponde.
30. Probar el endpoint de la API directamente (navegador, Postman o curl) para aislar si el problema es del frontend o del backend.

5.1 Tabla resumen de errores:

Código / Síntoma	Capa	Causa típica	Sección
502 / 503	Servidor	Cold start de Render o backend caído	3.1
401	Servidor	JWT inválido, expirado o ausente	3.2
403 al crear Usina	Servidor	Falta header Authorization en crearUsina.js	3.3
"No hay datos disponibles"	Servidor/Datos	Filtro o ID no coincide con contenido en Strapi	3.4
TypeError: Cannot read properties of undefined	Frontend	Falta validación de datos anidados de la API	4.1
Usuario "deslogueado" pese a login exitoso	Frontend	Clave inconsistente en localStorage (jwt vs token)	4.2
El texto editado no se refleja al instante	Frontend	handleSave.js devuelve campo fijo en vez de dinámico	4.3
Parpadeo / carga lenta de secciones	Frontend	Fetches independientes sin estado de carga global	4.4
next: not found / npm install falla	Entorno	Dependencias no instaladas / sin acceso a la registry	4.5
Imágenes rotas	Frontend/Datos	URL relativa de Strapi sin dominio anteponer	4.6

5.2 Identificadores de contenido esperados por el frontend:

Estos valores deben existir exactamente así en el backend de Strapi para que la home funcione sin errores:

- textos: texto-introduccion, texto-introduccion2
- acordeons: carreras
- imagens: logo

6. Recomendaciones generales de mantenimiento:

- Agregar manejo de errores (try/catch + verificación de res.ok) en todos los fetch, en particular en document.js, que actualmente no valida la respuesta antes de leer el JSON.
- Versionar un archivo .env.example (sin valores reales) para que cualquier persona que clone el proyecto sepa qué variables debe configurar.
- Unificar el manejo de autenticación en un solo módulo (hoy está repartido entre auth.js, useUser.js, checkRole.js y componentes individuales que leen localStorage directamente).
- Considerar un estado de error visible para el usuario final (hoy varios errores solo se loguean en consola con console.error, y la interfaz no informa nada o muestra mensajes genéricos).

Documentación de Actualización de Módulos

Objetivo: Describir el procedimiento para actualizar los módulos y dependencias del sistema, garantizando su correcto funcionamiento, seguridad y compatibilidad.

Requisitos Previos:

- Node.js v18.19.1 o superior.
- npm v9.2.0 o superior.
- Acceso al código fuente del proyecto.
- Respaldo de la información y configuraciones.

Módulos Utilizados:

- axios ^1.9.0
- next 15.5.9
- react ^19.0.0
- react-dom ^19.0.0
- react-hot-toast ^2.6.0
- react-icons ^5.5.0
- react-markdown ^10.1.0
- react-slick ^0.30.3
- slick-carousel ^1.8.1

Procedimiento de Actualización:

31. Verificar módulos desactualizados:

```
npm outdated
```

32. Actualizar dependencias:

```
npm update
```

33. Actualizar un módulo específico:

```
npm install nombre-del-paquete@latest
```

34. Reinstalar dependencias:

```
npm install
```

35. Verificar compilación:

```
npm run build
```

36. Reiniciar servicios:

```
pm2 restart all
```

37. Verificar estado:

```
pm2 status
```

38. Revisar registros:

```
pm2 logs --lines 20
```

Validación:

- Verificar acceso a todas las páginas.
- Comprobar formularios y galerías.
- Revisar que no existan errores en consola.

Conclusión: La actualización periódica de módulos permite mantener el sistema estable, seguro y compatible con nuevas tecnologías.

Manual Funcional — Sistema Web EAV Martín Malharro

1. Introducción:

Este manual funcional tiene como objetivo describir el funcionamiento general del sistema web que se desarrolló para la Escuela de Artes Visuales Martín A. Malharro.

2. Tecnologías utilizadas:

Tecnología	Función
Next.js	Framework frontend
React	Interfaz de usuario
Node.js	Entorno de ejecución
PostgreSQL	Base de datos
Strapi	CMS y API
PM2	Gestión de procesos
GNU/Linux	Sistema operativo servidor
Google OAuth	Autenticación externa

3. Arquitectura general del sistema:

El sistema utiliza una arquitectura cliente-servidor.

Funcionamiento general:

39. El usuario accede al sistema desde el navegador web.
40. El frontend desarrollado con Next.js renderiza la interfaz.
41. El frontend realiza solicitudes HTTP a la API.
42. Strapi administra el contenido dinámico.
43. PostgreSQL almacena la información persistente.

4. Estructura general del proyecto:

- Docs/: Contiene documentos, informes y archivos técnicos del proyecto.
- Public/: Contiene archivos públicos accesibles desde el navegador.
- Src/: Contiene el código fuente principal del sistema.
- Src/app/: Núcleo principal de la aplicación desarrollado con Next.js.

- Componentes/: Contiene componentes reutilizables en distintas partes del sistema.
- Hooks/: Funciones reutilizadas de lógica personalizada.
- Login/: Módulos relacionados con autenticación de usuarios.
- Validación/: Módulos encargados del control de permisos y validaciones.
- Styles/: Hojas de estilo generales de la aplicación.

5. Página principal:

Archivo principal: Page.js. Funciones: Mostrar contenido institucional, centralizar navegación, permitir acceso a las distintas secciones del sistema.

6. Sistema de autenticación:

Archivos relacionados: Auth.js, iniciarSesion.js, registrar.js, loginWithGoogle.js.

El sistema permite: registro de usuarios, inicio de sesión tradicional, autenticación mediante Google OAuth.

Funcionamiento: el usuario ingresa credenciales, el sistema valida la información, se genera un token de autenticación, el usuario accede al contenido protegido.

Observaciones detectadas durante el análisis técnico:

- Inconsistencias entre el almacenamiento de "token" y "jwt".
- Validaciones incompletas desde el lado del servidor.
- Exposición pública del token administrativo de Strapi.

7. Gestión de usuarios: Archivo: useUser.js. Funciones: Administración de sesión, persistencia de usuario, manejo del estado autenticado.

8. Sistema de roles y permisos: Archivos: checkRole.js, handleSave.js. Funciones: Validación de permisos, verificación de roles, control de acceso, validación de operaciones.

9. Componentes principales del sistema:

Agenda (agenda.js): Permite visualizar eventos y actividades institucionales.

Carrusel (carrusel.js): Permite mostrar contenido visual dinámico mediante rotación automática.

Imágenes (imagen.js, imagenById.js): Permiten obtener y renderizar imágenes desde la API.

Textos (text.js, textoById.js): Permiten obtener y mostrar contenido textual dinámico.

Documentos (document.js): Permite visualizar documentos institucionales.

Usina (usina.js, crearUsina.js, usinaProtegida.js): visualizar publicaciones, crear contenido dinámico, restringir acceso mediante autenticación.

10. Navegación y Layout: Archivos: Layout.js, userMenu.js. Funciones: Mantener estructura visual compartida, administrar navegación, generar menú interactivo de usuario.

11. Configuración general: Archivos: Config.js. Función: Centralizar funciones generales del sistema y almacenar rutas y variables utilizadas por la aplicación.

12. API y base de datos: El sistema utiliza Strapi como API principal para administrar contenido dinámico. La información almacenada en PostgreSQL es consumida desde el frontend mediante solicitudes HTTP. El sistema obtiene desde la API: textos, imágenes, agenda, documentos y publicaciones.

13. Scripts disponibles:

Script	Función
npm run dev	Inicia el entorno de desarrollo completo mediante el script scripts/dev.js.
npm run dev:frontend	Inicia únicamente el frontend Next.js en el puerto 3000.
npm run dev:backend	Inicia únicamente el backend ubicado en la carpeta backend.
npm run build	Genera la versión compilada para producción.
npm run start	Ejecuta la versión compilada del proyecto.
npm run lint	Analiza el código y detecta posibles errores o malas prácticas.